USBD45: Add a New Line — Sprint 3

Este documento descreve a implementação, configuração de dados e testes realizados para a **USBD45**.

1. Descrição da User Story

As a Planner, I want to add a new line. A line must have at least one segment.

Critérios de Aceitação e Regras de Negócio

- Uma linha é identificada por um **ID único** e deve ter **pelo menos um segmento** associado no momento da criação.

Validação de Integridade (Foreign Keys)

- O proprietário (**Owner VAT**) deve existir na base de dados.
- As instalações de início e fim (**Facilities**) devem existir na base de dados.
- A bitola (**Gauge**) deve existir na base de dados.

Validação de Topologia

- A instalação de início e a de fim **não pode ser a mesma**.

Validação de Domínio

- O campo `electrified` aceita apenas os valores **'Yes'** ou **'No'**.
- O número de vias (`number_tracks`) deve ser **1 (via única)** ou **2 (via dupla)**.
- O comprimento (`length_m`) e o peso máximo (`max_weight_kg_m`) devem ser **valores positivos (> 0)**.

Regras de Siding (Desvio)

- Se um siding for especificado, **tanto a posição (`siding_position`) como o comprimento (`siding_length`) devem ser fornecidos**.
- A posição do siding deve estar compreendida entre **0 e o comprimento total do segmento**.
- O comprimento do siding deve ser **positivo**.

Implementação

- A implementação é realizada através de uma **função PL/SQL**.

---

2. Explicação do Script (Função PL/SQL)

A lógica central da USBD45 foi encapsulada na função **`add_new_line`**. Como uma linha não pode existir sem segmentos, esta função realiza uma **inserção atómica**:

1. Criação do registo na tabela **Line**.
2. Inserção do primeiro segmento na tabela **Segment**.

Detalhes da Implementação

- **Assinatura da Função**: Recebe todos os atributos da linha e do primeiro segmento. Retorna uma `VARCHAR2` com mensagem de sucesso ou erro.
- **Verificação de Duplicados**: Valida se o `p_line_id` ou o `p_segment_id` já existem.
- **Validação de Referências**: Confirma a existência do Proprietário, Instalações e Bitola.
- **Validações de Regras de Negócio**:
    - Impede que `start_facility_id` seja igual a `end_facility_id`.
    - Valida manualmente *check constraints* (`electrified` , `number_tracks`).
    - Valida a consistência matemática dos dados de siding.
- **Inserção Transacional**: Insere a linha e o segmento (`sequence_order = 1`).
- **Gestão de Erros**: Executa `ROLLBACK` em caso de exceção para evitar dados órfãos.

Código: `usbd45-script.sql`

```
 -- usbd45-script.sql
-- ------------------------------------------------------------
-- USBD45: Function to add a new line with a mandatory segment
-- ------------------------------------------------------------

CREATE OR REPLACE FUNCTION add_new_line(
```

```
CREATE OR REPLACE FUNCTION add_new_line(
    p_line_id IN NUMBER,
    p_line_name IN VARCHAR2,
    p_owner_vat IN VARCHAR2,
    p_start_facility_id IN NUMBER,
    p_end_facility_id IN NUMBER,
    p_gauge IN NUMBER,
    -- Segment parameters (at least one segment must be provided)
    p_segment_id IN NUMBER,
    p_electrified IN CHAR,
    p_max_weight_kg_m IN NUMBER,
    p_length_m IN NUMBER,
    p_number_tracks IN NUMBER,
    p_siding_position IN NUMBER DEFAULT NULL,
    p_siding_length IN NUMBER DEFAULT NULL
) RETURN VARCHAR2
IS
    v_owner_count NUMBER;
    v_start_facility_count NUMBER;
    v_end_facility_count NUMBER;
    v_gauge_count NUMBER;
    v_line_exists NUMBER;
    v_segment_exists NUMBER;
    v_result VARCHAR2(500);
BEGIN
    SELECT COUNT(*) INTO v_line_exists FROM Line WHERE id = p_line_id;
    IF v_line_exists > 0 THEN
        RETURN 'ERROR: Line ID ' || p_line_id || ' already exists';
    END IF;

    SELECT COUNT(*) INTO v_segment_exists FROM Segment WHERE id = p_segment_id;
    IF v_segment_exists > 0 THEN
        RETURN 'ERROR: Segment ID ' || p_segment_id || ' already exists';
    END IF;

    SELECT COUNT(*) INTO v_owner_count FROM Line_Owner WHERE vat_number = p_owner_vat;
    IF v_owner_count = 0 THEN
        RETURN 'ERROR: Owner with VAT ' || p_owner_vat || ' does not exist';
    END IF;

    SELECT COUNT(*) INTO v_start_facility_count FROM Facility WHERE id = p_start_facility_id;
    IF v_start_facility_count = 0 THEN
        RETURN 'ERROR: Start facility with ID ' || p_start_facility_id || ' does not exist';
    END IF;

    SELECT COUNT(*) INTO v_end_facility_count FROM Facility WHERE id = p_end_facility_id;
    IF v_end_facility_count = 0 THEN
        RETURN 'ERROR: End facility with ID ' || p_end_facility_id || ' does not exist';
    END IF;

    SELECT COUNT(*) INTO v_gauge_count FROM Railway_Gauge WHERE gauge_value = p_gauge;
    IF v_gauge_count = 0 THEN
        RETURN 'ERROR: Gauge ' || p_gauge || ' does not exist in Railway_Gauge table';
    END IF;

    IF p_start_facility_id = p_end_facility_id THEN
        RETURN 'ERROR: Start and end facilities must be different';
    END IF;

    IF p_electrified NOT IN ('Yes', 'No') THEN
        RETURN 'ERROR: Electrified must be ''Yes'' or ''No''';
    END IF;

    IF p_length_m <= 0 THEN
        RETURN 'ERROR: Segment length must be greater than 0';
    END IF;
```

```
    IF p_max_weight_kg_m <= 0 THEN
        RETURN 'ERROR: Max weight per meter must be greater than 0';
    END IF;

    IF p_number_tracks NOT IN (1, 2) THEN
        RETURN 'ERROR: Number of tracks must be 1 (single) or 2 (double)';
    END IF;

    IF (p_siding_position IS NOT NULL AND p_siding_length IS NULL) OR
       (p_siding_position IS NULL AND p_siding_length IS NOT NULL) THEN
        RETURN 'ERROR: If siding is specified, both position and length must be provided';
    END IF;

    IF p_siding_position IS NOT NULL AND (p_siding_position < 0 OR p_siding_position > p_length_m) THEN
        RETURN 'ERROR: Siding position must be between 0 and segment length';
    END IF;

    IF p_siding_length IS NOT NULL AND p_siding_length <= 0 THEN
        RETURN 'ERROR: Siding length must be greater than 0';
    END IF;

    INSERT INTO Line (id, name, owner_vat, start_facility_id, end_facility_id, gauge)
    VALUES (p_line_id, p_line_name, p_owner_vat, p_start_facility_id, p_end_facility_id, p_gauge);

    INSERT INTO Segment (
        id, line_id, sequence_order, electrified, max_weight_kg_m,
        length_m, number_tracks, siding_position, siding_length
    ) VALUES (
        p_segment_id, p_line_id, 1, p_electrified, p_max_weight_kg_m,
        p_length_m, p_number_tracks, p_siding_position, p_siding_length
    );

    COMMIT;

    RETURN 'SUCCESS: Line ' || p_line_id || ' (' || p_line_name || ') created with segment ' || p_segment_id;

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RETURN 'ERROR: ' || SQLERRM;
END add_new_line;
/
```

---

3. Explicação de Cada Teste

Foram desenvolvidos **5 ficheiros de teste** para validar cenários de sucesso e erro.

Teste 1: Inserção Bem-Sucedida

Objetivo: Validar inserções válidas.

- Via dupla sem siding
- Via simples com siding
- Linha não eletrificada

Resultado Esperado: Mensagens iniciadas por **SUCCESS**.

```
 -- usbd45_test1.sql
SET SERVEROUTPUT ON;

-- 1.1: Via dupla sem siding
DECLARE
v_result VARCHAR2(500);
BEGIN
```

```
    DBMS_OUTPUT.PUT_LINE('===== Test 1.1: Add line with double track (no siding) =====');
    v_result := add_new_line(
        p_line_id => 100,
        p_line_name => 'Ramal Porto-Barcelos',
        p_owner_vat => 'PT503933813',
        p_start_facility_id => 5,
        p_end_facility_id => 8,
        p_gauge => 1668,
        p_segment_id => 1000,
        p_electrified => 'Yes',
        p_max_weight_kg_m => 8000,
        p_length_m => 15000,
        p_number_tracks => 2,
        p_siding_position => NULL,
        p_siding_length => NULL
    );
    DBMS_OUTPUT.PUT_LINE(v_result);
    DBMS_OUTPUT.PUT_LINE('');
END;
/

-- 1.2: Via simples com siding
DECLARE
v_result VARCHAR2(500);
BEGIN
    DBMS_OUTPUT.PUT_LINE('===== Test 1.2: Add line with single track and siding =====');
    v_result := add_new_line(
        p_line_id => 101,
        p_line_name => 'Ramal Midões-São Frutuoso',
        p_owner_vat => 'PT503933813',
        p_start_facility_id => 10,
        p_end_facility_id => 15,
        p_gauge => 1668,
        p_segment_id => 1001,
        p_electrified => 'Yes',
        p_max_weight_kg_m => 6400,
        p_length_m => 8000,
        p_number_tracks => 1,
        p_siding_position => 4000,
        p_siding_length => 500
    );
    DBMS_OUTPUT.PUT_LINE(v_result);
    DBMS_OUTPUT.PUT_LINE('');
END;
/

-- 1.3: Linha não eletrificada
DECLARE
v_result VARCHAR2(500);
BEGIN
    DBMS_OUTPUT.PUT_LINE('===== Test 1.3: Add non-electrified line =====');
    v_result := add_new_line(
        p_line_id => 102,
        p_line_name => 'Ramal Nine-Braga',
        p_owner_vat => 'PT503933813',
        p_start_facility_id => 20,
        p_end_facility_id => 30,
        p_gauge => 1668,
        p_segment_id => 1002,
        p_electrified => 'No',
        p_max_weight_kg_m => 7000,
        p_length_m => 12000,
        p_number_tracks => 1
    );
    DBMS_OUTPUT.PUT_LINE(v_result);
```

```
    DBMS_OUTPUT.PUT_LINE('');
END;
/
```

Teste 2: Verificação de ID Duplicado

Objetivo: Impedir reutilização de IDs de linha ou segmento.

Resultado Esperado: Mensagens de erro apropriadas.

```
 -- usbd45_test2.sql
SET SERVEROUTPUT ON;

-- Setup: Create a line that will be used to test duplicates
DECLARE
v_result VARCHAR2(500);
BEGIN
DBMS_OUTPUT.PUT_LINE('===== SETUP: Creating initial line for duplicate tests =====');
v_result := add_new_line(200, 'Original Line', 'PT503933813', 5, 8, 1668, 2000, 'Yes', 8000, 10000, 2);
DBMS_OUTPUT.PUT_LINE(v_result);
DBMS_OUTPUT.PUT_LINE('');
END;
/

-- 2.1: Duplicate Line ID
DECLARE
v_result VARCHAR2(500);
BEGIN
DBMS_OUTPUT.PUT_LINE('===== Test 2.1: Duplicate Line ID (should fail) =====');
v_result := add_new_line(200, 'Duplicate Line', 'PT503933813', 10, 15, 1668, 2001, 'Yes', 8000, 10000, 2);
DBMS_OUTPUT.PUT_LINE(v_result);
DBMS_OUTPUT.PUT_LINE('');
END;
/

-- 2.2: Duplicate Segment ID
DECLARE
v_result VARCHAR2(500);
BEGIN
DBMS_OUTPUT.PUT_LINE('===== Test 2.2: Duplicate Segment ID (should fail) =====');
v_result := add_new_line(201, 'New Line with Duplicate Segment', 'PT503933813', 10, 15, 1668, 2000, 'Yes', 8000, 10000, 2);
DBMS_OUTPUT.PUT_LINE(v_result);
DBMS_OUTPUT.PUT_LINE('');
END;
/
```

Teste 3: Validação de Foreign Keys

Objetivo: Garantir existência das entidades referenciadas.

- Proprietário inexistente
- Instalações inválidas
- Bitola inexistente

```
 -- usbd45_test3.sql
SET SERVEROUTPUT ON;

-- 3.1: Non-existent owner
DECLARE
v_result VARCHAR2(500);
BEGIN
    DBMS_OUTPUT.PUT_LINE('===== Test 3.1: Non-existent owner (should fail) =====');
    v_result := add_new_line(300, 'Line with Invalid Owner', 'PT999999999', 5, 8, 1668, 3000, 'Yes', 8000, 10000, 2);
    DBMS_OUTPUT.PUT_LINE(v_result);
    DBMS_OUTPUT.PUT_LINE('');
END;
/

-- 3.2: Non-existent start facility
DECLARE
v_result VARCHAR2(500);
BEGIN
    DBMS_OUTPUT.PUT_LINE('===== Test 3.2: Non-existent start facility (should fail) =====');
    v_result := add_new_line(301, 'Line with Invalid Start Facility', 'PT503933813', 9999, 8, 1668, 3001, 'Yes', 8000, 10000, 2);
    DBMS_OUTPUT.PUT_LINE(v_result);
    DBMS_OUTPUT.PUT_LINE('');
END;
/
-- (Os testes 3.3 e 3.4 seguem a mesma lógica para end_facility e gauge)
```

Teste 4: Validação de Regras de Negócio

Objetivo: Testar restrições lógicas.

- Instalações iguais
- Eletrificação inválida
- Comprimento ou peso inválidos

```
 -- usbd45_test4.sql
SET SERVEROUTPUT ON;

-- 4.1: Same start and end facility
DECLARE
v_result VARCHAR2(500);
BEGIN
    DBMS_OUTPUT.PUT_LINE('===== Test 4.1: Same start/end facility (should fail) =====');
    v_result := add_new_line(400, 'Line with Same Facilities', 'PT503933813', 5, 5, 1668, 4000, 'Yes', 8000, 10000, 2);
    DBMS_OUTPUT.PUT_LINE(v_result);
    DBMS_OUTPUT.PUT_LINE('');
END;
/

-- 4.2: Invalid electrified value
DECLARE
v_result VARCHAR2(500);
BEGIN
    DBMS_OUTPUT.PUT_LINE('===== Test 4.2: Invalid electrified value (should fail) =====');
    v_result := add_new_line(401, 'Line with Invalid Electrified', 'PT503933813', 5, 8, 1668, 4001, 'Maybe', 8000, 10000, 2);
    DBMS_OUTPUT.PUT_LINE(v_result);
    DBMS_OUTPUT.PUT_LINE('');
END;
/

-- 4.4: Zero length
DECLARE
v_result VARCHAR2(500);
BEGIN
    DBMS_OUTPUT.PUT_LINE('===== Test 4.4: Zero segment length (should fail) =====');
    v_result := add_new_line(403, 'Line with Zero Length', 'PT503933813', 5, 8, 1668, 4003, 'Yes', 8000, 0, 2);
    DBMS_OUTPUT.PUT_LINE(v_result);
    DBMS_OUTPUT.PUT_LINE('');
END;
/
```

Teste 5: Validação de Siding

Objetivo: Garantir consistência dos dados de siding.

- Dados parciais
- Valores fora dos limites

```
-- usbd45_test5.sql
SET SERVEROUTPUT ON;

-- 5.1: Siding position without length
DECLARE
v_result VARCHAR2(500);
BEGIN
    DBMS_OUTPUT.PUT_LINE('===== Test 5.1: Siding position without length (should fail) =====');
    v_result := add_new_line(500, 'Line with Incomplete Siding 1', 'PT503933813', 10, 15, 1668, 5000, 'Yes', 6400, 8000, 1, 4000, NU
    DBMS_OUTPUT.PUT_LINE(v_result);
    DBMS_OUTPUT.PUT_LINE('');
END;
/

-- 5.3: Siding position > segment length
DECLARE
v_result VARCHAR2(500);
BEGIN
    DBMS_OUTPUT.PUT_LINE('===== Test 5.3: Siding position > segment length (should fail) =====');
    v_result := add_new_line(502, 'Line with Invalid Siding Position', 'PT503933813', 10, 15, 1668, 5002, 'Yes', 6400, 8000, 1, 9000
    DBMS_OUTPUT.PUT_LINE(v_result);
    DBMS_OUTPUT.PUT_LINE('');
END;
/
```